

Socratic AI编程辅导平台完整落地方案 (LangChain+LangGraph多智能体)

Socratic AI 编程辅导平台完整落地方案 (LangChain+LangGraph 多智能体)

文档里的需求核心：不直接生成代码、苏格拉底启发式对话、自调节学习四阶段（计划 / 监测 / 评估 / 反思）、5 个专用 Agent、4 级分层 Hint、循环式编程 workflow。

LangChain 负责封装 LLM、提示词、记忆、代码解析组件；LangGraph 做全局状态流转、多智能体调度、分支 / 循环 / 断点流程。下面分整体架构→状态定义→5 个 Agent 节点→业务流程图→Hint 分级机制→前端模块→代码最小 Demo→部署步骤完整说明怎么做。

一、整体技术分层

1) 底层组件层 (LangChain)

全部复用 LangChain 现成模块，不用重复封装：

1. LLM 封装：GPT-4o/Claude，独立每个 Agent 专属 System Prompt
2. 对话记忆：LangGraph MessagesState 持久化多轮对话、学生代码、计划、错题
3. 代码解析工具：PythonCodeInterpreter、语法错误检测、变量推演工具
4. 提示词模板库：苏格拉底提问模板、4 级 Hint 模板、反思问卷模板
5. 结构化输出：Pydantic 强制 Agent 输出结构化判断（学生困惑等级、错误类型、当前学习阶段）

2) 流程调度层 (LangGraph 核心)

- 用 StateGraph 搭建全局 workflow，所有 Agent 作为独立节点
- 共享统一全局 State 存储全部会话数据，Agent 之间不直接通信，靠状态总线传数据
- 条件分支实现：学生卡顿 / 答错 / 完成 / 请求提示 四种不同路由逻辑
- 循环链路：编码→监测 Debug→再编码；任务完成后进入反思阶段终止

3) 业务应用层 (网站前端)

三大页面模块：题库出题模块、对话辅导模块、学习反思复盘模块

二、第一步：定义全局共享 State (LangGraph 核心)

所有 Agent 读写同一份状态，承载文档全部业务数据，用 TypedDict 定义：

代码块

```
1  from typing import TypedDict, List, Optional
2  from langchain_core.messages import BaseMessage
3
4  class TutorState(TypedDict):
5      # 基础会话
6      user_id: str
7      problem_content: str          # 当前编程题目（低/中/高难度）
8      difficulty: str
9      messages: List[BaseMessage] # 完整对话记录
10
11     # 学生产出
12     plan_form: dict                # 学生填写的计划：输入/输出/步骤
13     student_code: str              # 学生提交代码
14     code_prediction: str           # 学生对代码运行结果预判
15     student_reflection: dict       # 完成后反思回答
16
17     # AI多智能体输出缓存
18     understanding_score: int        # 问题理解度0-10
19     confusion_level: int            # 困惑等级0-3（对应Hint 0~3级）
20     code_error_type: Optional[str] # 语法/逻辑/计算/循环错误
21     socratic_question: str         # 本轮苏格拉底提问
22     hint_level: int                # 当前发放Hint等级
23     learning_summary: str          # 最终学习总结
24
25     # 流程标记（控制路由）
26     current_phase: str              # plan / coding / monitor / debug / reflect
27     is_stuck: bool                  # 是否卡顿（1分钟无回复/重复提问）
28     task_finished: bool            # 是否完成整道题
```

三、第二步：5 个专用 Agent（对应文档 Multi-Agent）

每个 Agent 是 LangGraph 独立 Node，用 LangChain 封装专属 Prompt+LLM，各司其职：

Agent1: Problem Understanding Agent（ReAct）

功能：解析题目、评估学生理解程度、判断是否进入计划阶段

输入：state.problem_content + 学生初始回答

输出：understanding_score、补充完善学生的 Plan 表单

SOP：

1. 读取题目难度，拆解输入输出需求
2. 分析学生填写的 Plan，识别缺失步骤

3. 生成引导问题补全计划（Plan 阶段 AI 行为）
4. 标记学生理解度，决定是否进入编码环节

Agent2: Socratic Questioning Agent (CoT)

核心苏格拉底对话 Agent，禁止直接给答案

功能：根据当前阶段 + 学生错误生成启发式提问，管理 4 级 Hint 发放逻辑
强制 Prompt 约束（文档要求）：

1. 绝不直接给出代码 / 标准答案
2. 按计划→编码→监测→反思四阶段匹配提问模板
3. 学生卡顿 (is_stuck=True) 逐级提升 Hint 等级 (0→1→2→3)
4. 引导学生解释自己的推理，自行定位 bug

Agent3: Code Analysis Agent

功能：静态解析学生代码，定位错误类型、推演变量运行结果
能力：

1. 检测语法错误、逻辑错误、计算错误（如文档 average 除以 2）
2. 执行代码推演，对比学生 code_prediction 预判结果
3. 不输出报错，只输出错误类别、矛盾点，交给苏格拉底 Agent 提问

Agent4: Metacognitive Agent (元认知监测)

对应 Monitor 监测阶段

功能：全程跟踪学生思维过程，判断是否卡顿、重复犯错、元认知薄弱
判定学生 Struggle（卡顿）规则：

- 前端记录 1 分钟无回复 → is_stuck=True
- 学生反复询问同一知识点、多次同类错误 → confusion_level+1
行为：触发分级 Hint、追问元认知问题（“你觉得哪一步最难？”）

Agent5: Assessment Agent (评估 + 反思)

两个阶段工作

1. 编码完成后：联合评估代码正确性、总结错误策略
2. 进入 Reflect 反思模块：生成结构化反思问卷、自动输出 Learning Reflection Summary
输出：学习总结、知识点薄弱点、可迁移编程思路

四、第三步：LangGraph 完整业务流转图（贴合文档学习流程）

整体链路

START → ProblemUnderstandingAgent（Plan 计划阶段）
↓（条件路由）
理解不足 → 回到 Socratic 提问补全计划
计划完整 → 进入编码阶段
↓
学生提交代码 → CodeAnalysisAgent + MetacognitiveAgent 监测
↓
存在错误 / 预判不符 → SocraticAgent 发引导提问 + 分级 Hint（循环回到编码）
代码无错误 → AssessmentAgent 开启反思模块
↓
学生填写反思问卷 → Assessment 生成学习总结 → END

关键条件边（核心控制逻辑）

- 1. 计划阶段路由：理解度 < 5 分 → 继续苏格拉底提问；≥5 分放行编码
- 2. 编码监测路由：检测到 error /is_stuck=True → 提升 Hint 等级，返回对话；无错误进入反思
- 3. Hint 升级规则：每一次卡顿未解决，hint_level +=1，最高到 3 级（示例代码）
- 4. 反思阶段为终点，生成总结后流程结束

五、第四步：4 级分层 Hint 机制工程实现（文档要求）

由 Socratic Agent 根据 state ["hint_level"] 自动匹配，严格逐级释放，不跨级：

等级	类型	逻辑与示例
Level0 元认知提问	无技术提示，只问思考	What is the problem asking you to achieve?
Level1 概念提示	引导对应编程概念	需要重复执行操作，用什么编程结构？
Level2 技术提示	点出语法关键字	你可以尝试使用 for 循环结构
Level3 极简示例	仅最后兜底，少量示例代码	for i in range(5): print(i)

Prompt 内置 Hint 控制逻辑

代码块

```
1  你只能按当前hint_level发放提示：
2  hint_level=0：仅元认知问题；
3  hint_level=1：仅概念引导；
4  hint_level=2：点明语法结构；
```

- 5 hint_level=3: 提供极简代码片段。
- 6 学生未充分尝试前, 禁止直接跳高级Hint。

六、第五步：三大网站模块前后端对接

模块 1：出题模块（低 / 中 / 高三档）

- 前端：选择难度、题目数量，生成编程题存入 state.problem_content
- 后端：LangChain 封装题库 Prompt 批量生成标准化编程题（输入 / 输出 / 测试用例）

模块 2：AI Socratic 对话辅导核心页面

分三栏：

1. 左侧：计划填写表单（Problem Analysis/Input/Output/Possible steps）
2. 中间：代码编辑器（学生写代码 + 提交预测运行结果）
3. 右侧：AI 苏格拉底对话窗口
交互逻辑：
 4. 学生先填计划表单提交 → 流入 ProblemUnderstandingAgent
 5. 表单通过后解锁代码编辑器
 6. 提交代码自动触发代码解析 + 监测 Agent
 7. AI 回复永远是引导提问，不直接改代码（对比传统 AI 直接报错）

模块 3：Reflection 反思模块（任务完成后解锁）

前端渲染固定 4 类反思问题表单：

1. Understanding：最难的部分是什么
 2. Strategy：用了什么解题策略
 3. Debugging：如何发现并修复错误
 4. Transfer：该思路还能用在哪些场景
- 学生提交后，Assessment Agent 生成专属学习总结存入会话记录

七、最小可运行代码骨架（LangChain+LangGraph）

代码块

```
1 # 1. 导入依赖
2 from langgraph.graph import StateGraph, START, END
3 from langchain_openai import ChatOpenAI
4 llm = ChatOpenAI(model="gpt-4o", temperature=0.3)
```

```

5
6 # 2. 全局状态 (简化版)
7 class TutorState(TypedDict):
8     messages: List[BaseMessage]
9     problem_content: str
10    student_code: str
11    current_phase: str
12    hint_level: int
13    is_stuck: bool
14    task_finished: bool
15
16 # 3. 定义5个Agent节点函数 (示例只写一个)
17 def problem_understanding_agent(state: TutorState):
18     prompt = """你是Problem Understanding Agent, 苏格拉底编程导师, 评估学生对题目的
19     理解, 引导完善计划, 不直接给代码"""
20     resp = llm.invoke([("system", prompt)] + state["messages"])
21     return {"socratic_question": resp.content, "understanding_score": 7}
22
23 def socratic_guide_agent(state: TutorState):
24     # 内置四级Hint逻辑
25     hint_rules = ["元认知提问", "概念提示", "技术关键词", "极简代码示例"]
26     current_hint = hint_rules[state["hint_level"]]
27     prompt = f"遵循苏格拉底教学, 仅使用{current_hint}, 不直接给出答案"
28     resp = llm.invoke([("system", prompt)] + state["messages"])
29     return {"messages": state["messages"] + [resp], "socratic_question":
30     resp.content}
31
32 # 其余3个agent: code_analysis、metacognitive_monitor、assessment_reflect
33
34 # 4. 构建LangGraph图
35 builder = StateGraph(TutorState)
36 # 添加所有节点
37 builder.add_node("plan_agent", problem_understanding_agent)
38 builder.add_node("socratic_agent", socratic_guide_agent)
39 # 补充code、monitor、assessment节点...
40
41 # 入口
42 builder.add_edge(START, "plan_agent")
43 # 条件路由: 计划完成→编码; 未完成→苏格拉底提问
44 def route_plan(state):
45     if state["understanding_score"] >= 5:
46         return "code_agent"
47     return "socratic_agent"
48 builder.add_conditional_edges("plan_agent", route_plan)
49
50 # 编码阶段循环: 有错/卡顿回到对话, 无错进入反思
51 def route_coding(state):

```

```

50     if state["is_stuck"] or state["code_error_type"]:
51         return "socratic_agent"
52     return "assessment_agent"
53     builder.add_conditional_edges("code_agent", route_coding)
54
55     # 反思节点指向结束
56     builder.add_edge("assessment_agent", END)
57
58     # 编译运行
59     graph = builder.compile()
60     # 启动会话
61     res = graph.invoke({
62         "problem_content": "计算三名学生平均分",
63         "hint_level": 0,
64         "is_stuck": False,
65         "current_phase": "plan",
66         "messages": []
67     })

```

八、完整落地实施步骤（分 4 阶段）

阶段 1：提示词工程（1-2 天）

1. 为 5 个 Agent 分别编写独立 System Prompt，严格对齐文档苏格拉底约束
2. 制作 4 级 Hint 模板库、计划表单校验模板、反思问卷模板
3. 统一约束：禁止直接输出完整代码、强制启发式提问、元认知引导

阶段 2：LangChain 组件封装（2 天）

1. 封装 LLM 调用、代码静态解析、运行推演工具
2. 会话记忆封装，持久化每轮对话、代码、计划、Hint 等级
3. 结构化输出：用 Pydantic 让 Agent 固定返回理解分、错误类型、困惑等级

阶段 3：LangGraph 多智能体流程搭建（3-4 天，核心）

1. 定义完整 TutorState 全局状态
2. 实现 5 个 Agent 节点函数
3. 编写所有条件路由逻辑：计划分支、编码循环、Hint 升级、反思终点
4. 调试循环链路：学生反复出错时自动提升 Hint 等级，形成脚手架

阶段 4：前后端联调 + 网站模块开发（5-7 天）

1. 前端三个页面：出题、对话编辑器、反思复盘

2. 埋点逻辑：前端计时判断 is_stuck（1 分钟无交互标记卡顿）
3. 表单联动：计划表单、代码提交、反思问卷数据同步写入 LangGraph State
4. 会话持久化：保存整套流程数据用于后续教学评估、性别公平性研究（论文亮点）

九、对应文档三大亮点的技术实现说明

1. AI 是认知脚手架，不是代码生成器

通过 Socratic Agent Prompt 强约束 + 4 级 Hint 逐级释放，全程阻断完整代码输出，仅引导思考。

2. 非结构化 / 复杂编程问题求解

LangGraph 循环架构支持多轮反思、反复调试，适配开放式编程难题，而非一次性线性问答。

3. 多智能体分工解耦

5 个专用 Agent 拆分理解、提问、代码分析、元认知监测、评估五大职责，避免单一 LLM 上下文过载，教学行为更专业可控。

十、扩展优化方向（论文加分项）

1. RAG 题库知识库：存入不同难度编程题、常见编程误区，Agent 检索对应概念
2. 长期学生记忆：存储历史错题、薄弱知识点，个性化调整提问难度
3. 可视化 LangGraph 流程：展示学生完整解题路径，用于教学研究数据分析
4. 性别公平适配 Prompt：差异化引导话术，平衡不同性别学生交互体验（论文标题核心亮点）

需要我把**5 个 Agent 完整可直接复制的 System Prompt**整理给你吗？

（注：部分内容可能由 AI 生成）